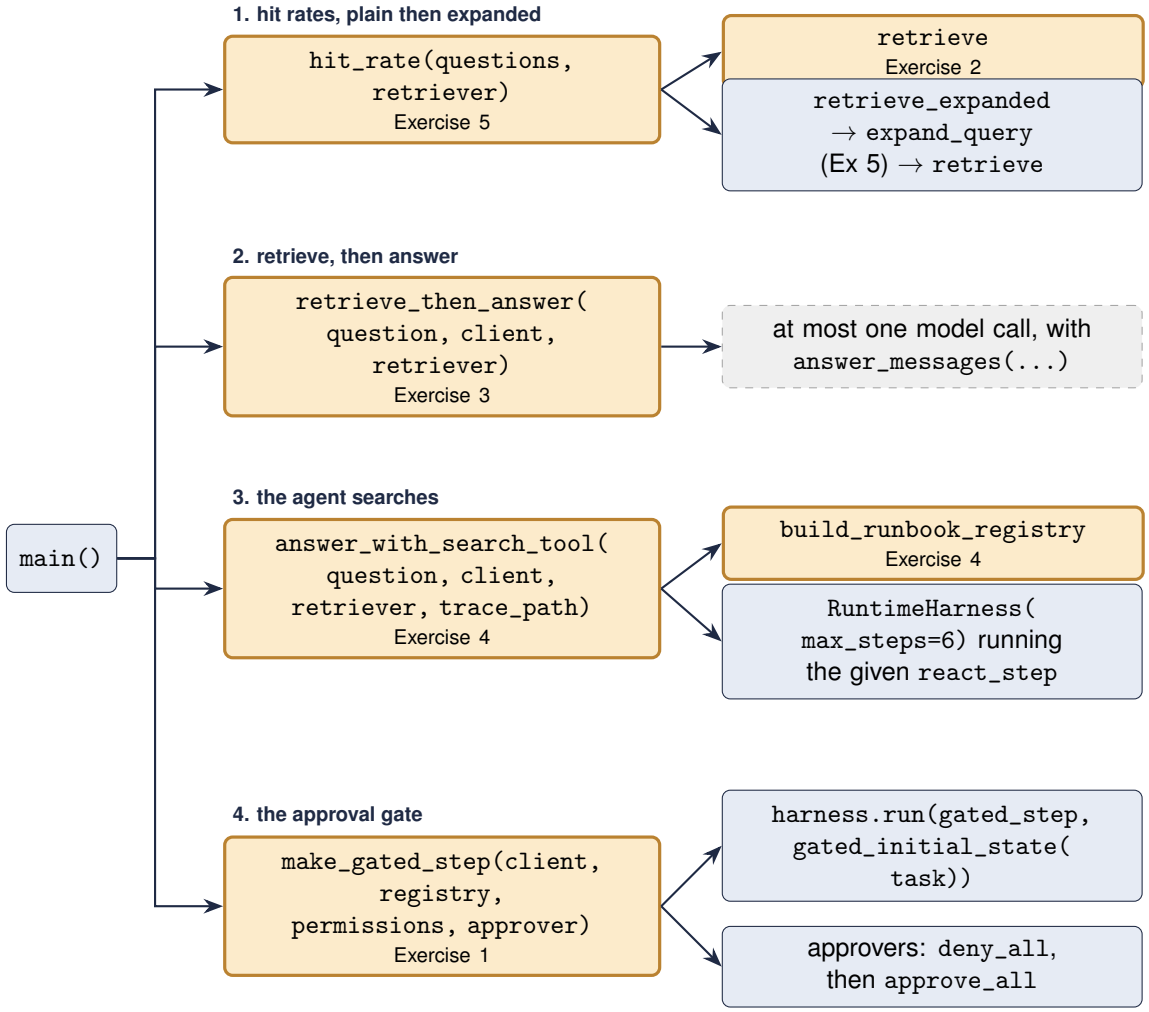
# Lab 8b call flow, 1 of 6: who calls whom

`python3 scaffold.py` runs four parts in order. Gold boxes are your exercises; everything else is given.




*`retrieve_expanded` is given: it only joins your two functions.*

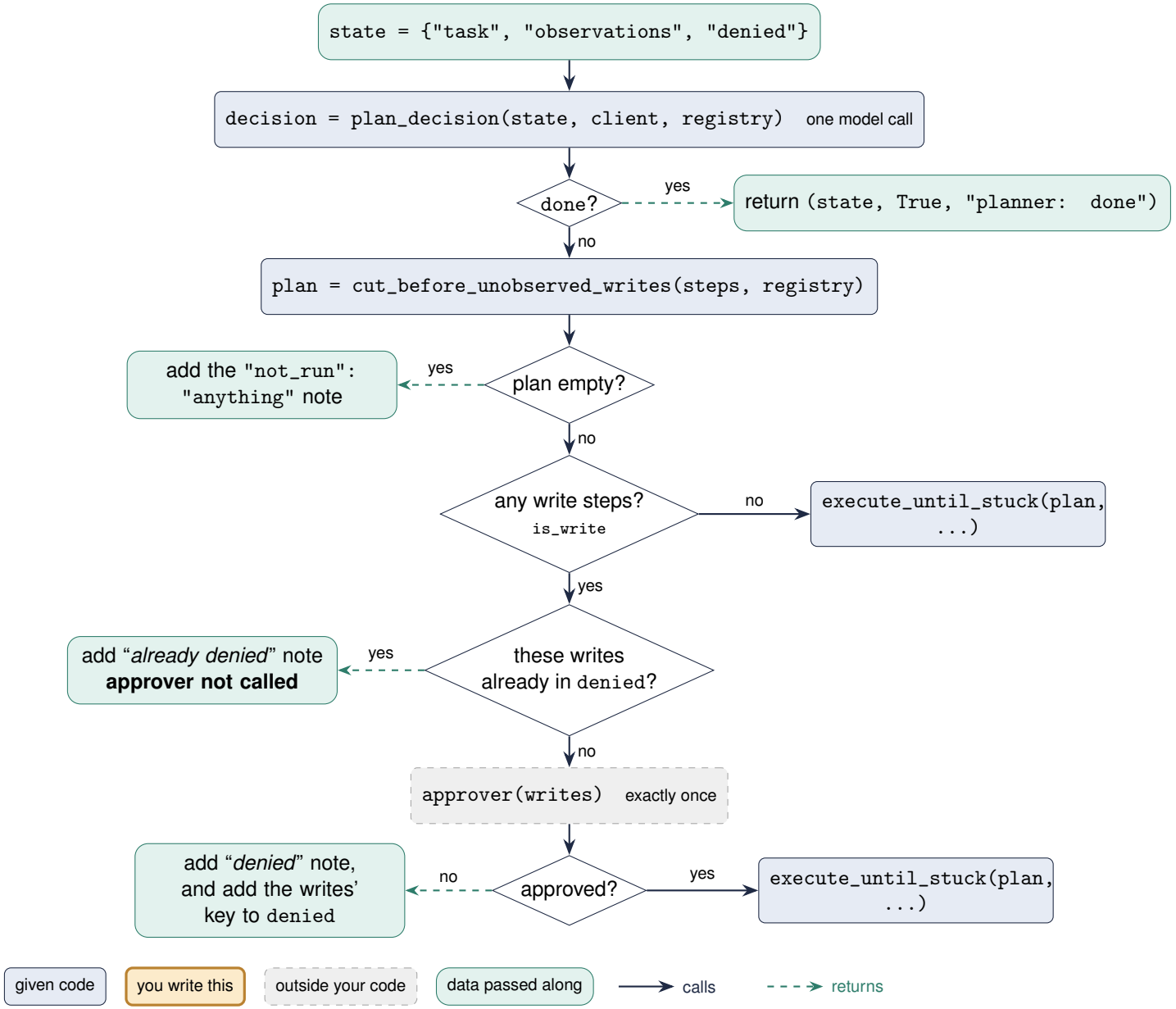
*`search_runbook`'s `fn` calls the `retriever` you pass in. That is how plain and expanded retrieval reach the agent.*

*Lab 8a's planner is given here: `plan_decision`, `cut_before_unobserved_writes`, `execute_until_stuck`, `is_write`. Your gate wraps them.*

given code | you write this | outside your code | data passed along | → calls | - - → returns

# Lab 8b call flow, 2 of 6: one round of the gated step

Exercise 1. `make_gated_step` returns this function; the harness calls it once per round with just `state`.




***The key*** *is* `json.dumps(writes, sort_keys=True)`*: the same writes always give the same text, so "seen before" is a plain string check.*

***Every path ends the same way:*** *return a* new *state (new observations, a new* `denied` *list),* `False`*, and the new observations. Only the* `done` *path returns* `True`*.*

***The approver only sees writes.*** *Reads never need permission, and the write rule already moved writes after the reads they depend on.*

# Lab 8b call flow, 3 of 6: the gate over a whole run

A real run, 29 September: the task from Lab 8a, with the reviewer `deny_all`. Ten rounds, one question to the reviewer.

| Round | The planner proposes | Reviewer asked? | Observation added |
|---|---|---|---|
| **1–2** | reads: `search_tickets`, then histories | no: reads need no approval | the results |
| **3** | `escalate_ticket` for 23 and 42 | **yes**, and denies | "A human reviewer denied these writes. Do not propose them again." |
| **4** | the same two escalations | **no**: key already in `denied` | "These writes were already denied ..." |
| **5** | more reads (ticket statuses) | no | the results |
| **6–10** | the same two escalations, every round | **no** | "already denied", each round |

`deny_all`: status `exhausted` after 10 rounds, escalated `[]`. The reviewer was asked **once**.

`approve_all`: status `done`, escalated `[23, 42]`. The reviewer was asked once and said yes.

***The model ignored the request.*** *Told not to propose the writes again, it proposed them again, seven more times. A rule you ask the model to follow is a suggestion.*

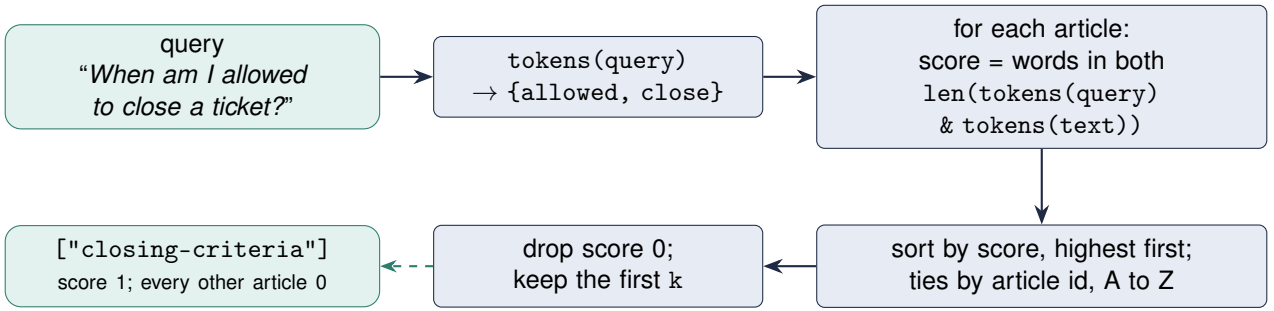
***The code did not.*** *The `denied` list in `state` made every repeat a string check. No escalation ran, and the reviewer was not pestered.*

***What stopped the run*** *was the harness's round limit, not good sense. Read `trace_gate_deny_all.jsonl` after your own run and count.*

given code | you write this | outside your code | data passed along | → calls | - - → returns

# Lab 8b call flow, 4 of 6: inside `retrieve`

Exercise 2. Plain keyword overlap, computed with the given `tokens()` and `RUNBOOK`. Real numbers below.




| Query | `tokens(query)` | Result |
|---|---|---|
| When am I allowed to close a ticket? | allowed, close | `["closing-criteria"]` |
| A customer wants their money back. Who handles that? | back, handles, money, wants | `[]`: no article says any of these |
| refund process | process, refund | `[]`: the article says "refunds" |

***Where did "ticket", "customer" and "When" go?*** *`STOPWORDS` removes common words, and also words in nearly every article, which would match everything.*

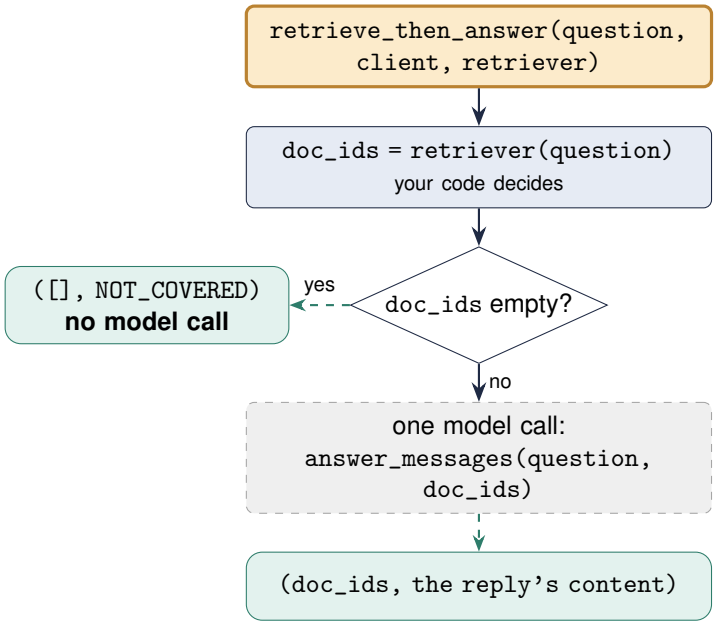
***Exact words only.*** *"refund" and "refunds" are different words here. There is no stemming, which is why the third query finds nothing.*

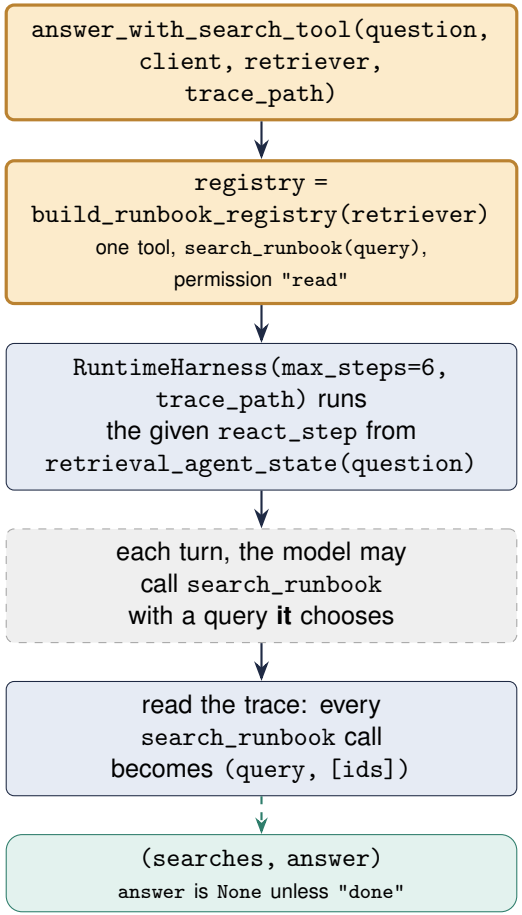
***An empty list is a real answer.*** *It means no article shares a single word with the query. Exercise 3 depends on it.*

given code | you write this | outside your code | data passed along | → calls | - - → returns

# Lab 8b call flow, 5 of 6: two ways to answer from the runbook

Exercise 3 on the left is a fixed workflow. Exercise 4 on the right hands the search to the agent.



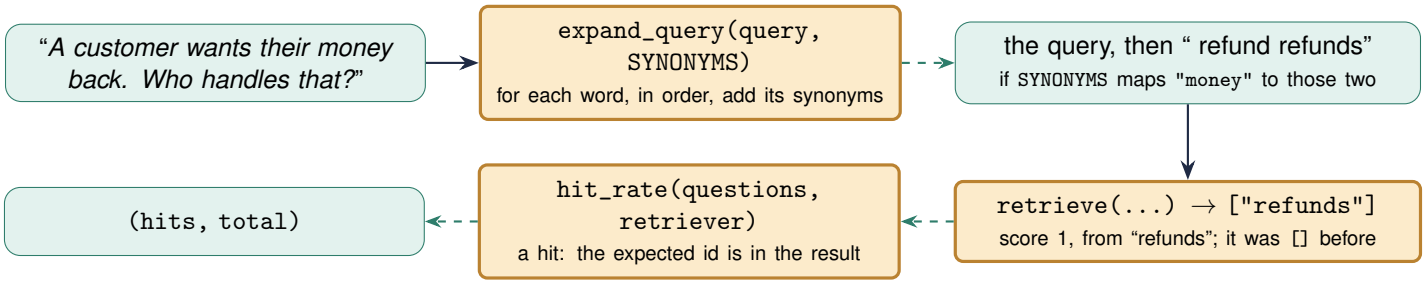




| Real run, 29 Sep | The agent searched for | Then |
|---|---|---|
| plain `retrieve` | "refund process" → `[]`, then "money back request" → `[]` | guessed an answer anyway |
| with synonyms | "refund process" → `[]`, then "money back request" → `["refunds"]` | answered from the refunds article |

given code · you write this · outside your code · data passed along · calls · returns

# Lab 8b call flow, 6 of 6: closing the vocabulary gap

Exercise 5. `expand_query` adds words; the given `retrieve_expanded` retrieves with them; `hit_rate` measures the result.




| What gets measured | Why |
|---|---|
| `questions.json`: 12 questions, plain `retrieve` finds 7 | your synonyms must raise this, and must not lose any of the 7 |
| hidden questions, never shipped to you | the graders' check that your synonyms are true in general, not patched for twelve sentences |

***Expansion only ever adds words.*** *The original question is kept whole, so a question that already worked still has every word it had.*

***Added words can help the wrong article too.*** *A synonym that appears in several articles raises them all. Check which article wins, not just whether yours scored.*

***Some misses are not about synonyms.*** *When one remains, look at which article it ties with and why. The tie-break is alphabetical.*

given code | you write this | outside your code | data passed along | → calls | - - → returns